Trace Data Quality for LLM-Agent Observability: A Controlled Study of Schema Loss and Diagnostic Degradation in Telemetry Backends

Krishna Chaitanya Balusu
Independent Researcher
San Francisco, CA, USA
krishnabkc15@gmail.com

***Abstract*—LLM-agent systems generate execution traces that look superficially similar to ordinary distributed traces but carry different diagnostic semantics: planning steps, reasoning loops, guardrail decisions, delegation, memory accesses, retrieval, tool calls, and LLM invocations need to remain distinguishable for the structural diagnostics studied here after collection, export, and downstream analysis. Existing observability schemas preserve some LLM and tool events, but often collapse or remove agent-orchestration events. This paper treats LLM-agent observability as a big-data quality problem. We define a trace-data quality model covering schema coverage, semantic preservation, joinability, payload-redacted structural sufficiency, and diagnostic-proxy sufficiency. Using existing open AgentTelemetry artifacts as measurement data and independent τ-bench trajectories, we analyze 2,148 traces containing 46,267 events exported as OpenTelemetry spans: 112 SWE-bench traces, a separate 56-trace end-to-end/topology suite, and 1,980 τ-bench tool-agent trajectories. We project those events into seven schemas, export each projection through the OpenTelemetry SDK into a SQLite trace backend, evaluate eight structural visibility queries, and run field ablations over the reference schema. We also validate an OTel Collector receiver/exporter round trip for the custom-attribute projection. LLM-focused projections preserve tool, token, and retrieval queries, but retain only 33.3–55.6% of semantic event kinds and answer 37.5–50.0% of positive structural visibility query families overall. They answer 0.0% of planning, reasoning-loop, memory, and delegation queries. Removing only parent identifiers drops query recall to 87.5% by breaking tool provenance; removing event-kind fields drops query recall to 0.0%. A payload-redacted structural projection retains 100.0% recall for measured structural visibility queries while removing prompt/completion/tool payload content. An OTel+custom-attributes projection shows the same structural recall is possible in generic span storage when non-content diagnostic fields are retained. The result is a reproducible controlled field-retention study of where LLM-agent trace pipelines lose structural answerability and which schema properties the measured queries require.**

***Index Terms*—big data, observability, LLM agents, data quality, tracing, OpenTelemetry, diagnostic analytics**

I. Introduction

Distributed tracing was designed to make large software systems observable by converting executions into structured event streams. That assumption becomes fragile for LLM-agent systems. An agent run is not just an RPC path or a sequence of service calls; it is a data-rich process involving planning, reasoning, retrieval, tool selection, guardrail checks, delegation, memory reads and writes, and repeated LLM invocations. If a telemetry pipeline preserves only generic spans, tokens, and tool calls, downstream analytics cannot distinguish a healthy multi-step plan from a reasoning loop, a blocked guardrail from an ordinary application branch, or a stale memory read from a normal lookup.

This is a data-quality problem. Agent traces are collected at high volume, transformed across incompatible schemas, exported into tracing backends, and queried by incident-response and analytics workflows. The core question is not only whether traces are present, but whether they retain the semantic fields required for later joins, aggregation, privacy filtering, and diagnosis. In big-data terms, LLM-agent observability raises a veracity and interoperability problem: the pipeline may contain many spans while losing the information that makes those spans analytically meaningful.

OpenTelemetry (OTel) [1] and its GenAI semantic conventions provide a foundation for LLM invocation and tool execution telemetry [2]. LLM-observability systems such as OpenLLMetry [3], LangSmith [4], Langfuse [5], and OpenInference [6] extend that ecosystem. However, the agent layer adds orchestration semantics that are not consistently preserved across schemas: planning, reasoning, guardrails, delegation, and memory. The practical effect is schema loss: trace rows continue to exist, but columns and event categories needed for diagnosis are missing, collapsed, or no longer joinable.

This paper asks how to measure that loss. We use the public AgentTelemetry artifacts from AIware 2026 [7], [8] as data sources, not as the novelty of this paper. The prior work introduced a benchmark, span taxonomy, fault classes, and toolkit. Here, the contribution is different: a data-engineering analysis of trace-data quality and diagnostic degradation across telemetry schemas. In particular, we ask:

- **RQ1**: What trace data are lost when LLM-agent executions are mapped into generic or LLM-only telemetry schemas?
- **RQ2**: How does schema loss affect downstream diagnostic analytics such as loop detection, root-cause localization, and spans to diagnosis?
- **RQ3**: Which schema properties are sufficient for payload-redacted structural queryability, and which calibration risks remain even when agent semantics are preserved?

The analysis combines three pieces of evidence. First, we perform a schema-interoperability study over 2,148 traces

containing 46,267 events exported as OpenTelemetry spans: 112 SWE-bench [9] traces, a separate 56-trace end-to-end/topology suite, and 1,980 τ-bench [10] tool-agent trajectories. The τ-bench workload provides independent task outcomes and expected tool-action labels, so two query families are evaluated against external task labels rather than agent event-kind labels. Second, we run eight downstream structural visibility queries against seven schema projections, measuring which answers survive schema transformation after export through the OpenTelemetry SDK into a SQLite trace backend. Third, we run field ablations that remove one class of fields at a time—event kind, parent identifiers, timestamps, memory fields, guardrail fields, token/cost fields, and content payloads. A 3,780-row controlled fault matrix is used only as validation that the schema distinction aligns with fault-family detection.

The results are concrete but deliberately scoped. LLM-focused projections preserve tool, token, and retrieval queries, but retain only 33.3–55.6% of semantic event kinds and answer only 37.5–50.0% of positive structural visibility query families overall. They answer 0.0% of planning, reasoning-loop, memory, and delegation queries because the required orchestration fields are not present. Field ablations show the failure modes are not all-or-nothing: dropping only parent identifiers breaks tool provenance, dropping only timestamps breaks reasoning-loop detection, and dropping only memory or guardrail attributes breaks the corresponding audits. Conversely, a payload-redacted structural projection preserves all measured structural visibility answers without retaining prompt, completion, reasoning-chain text, tool payload, or retrieved query content.

Contributions.

- A trace-data quality model for LLM-agent observability covering schema coverage, semantic preservation, joinability, payload-redacted structural sufficiency, and diagnostic sufficiency.
- A reproducible schema-interoperability study over 2,148 traces and 46,267 source events materialized as 740,272 backend span rows across projections, separating row retention, semantic-kind coverage, required-attribute preservation, join-key preservation, and query answerability.
- Empirical evidence that schema loss degrades downstream structural diagnostic answerability: LLM-focused projections answer 37.5–50.0% of positive structural visibility query families overall and 0.0% of the measured planning, reasoning-loop, memory, and delegation families.
- A separate τ-bench action-visibility control that measures expected-action and argument-shape visibility, then shows that a missing-action proxy is weak against task pass/fail baselines.
- A custom-attribute projection showing that generic OTel-style storage can preserve structural queryability when non-content diagnostic fields are retained.
- A field-ablation study showing which individual field classes are load-bearing for reasoning-loop, guardrail, memory, delegation, tool-provenance, token-accounting, and retrieval-grounding queries.
- Generated tables and scripts that rederive the paper's metrics from sanitized derived artifacts without running new LLM experiments.

II. Background and Boundary

A. Trace data in LLM-agent systems

Classical distributed tracing systems such as Dapper, X-Trace, and Magpie model request execution as structured event streams across services [11]–[13]. A span typically has a name, timestamp, duration, parent, trace identifier, service attributes, and semantic-convention fields. That model is still useful for LLM-agent systems, but it is not enough. A single agent trace may contain an LLM call whose output is a plan, a retrieval call that grounds the plan, a tool call selected by a reasoning step, a guardrail check, and memory updates used by later turns. These are not interchangeable internal spans. They define the analytical schema of the trace.

OTel GenAI conventions standardize parts of this schema, especially LLM invocation and tool execution [2]. OpenInference and OpenLLMetry-style systems extend the space with LLM and retrieval observability [3], [6]. Yet agent failures reported in recent work include reasoning loops, delegation cycles, planning failures, memory corruption, and guardrail bypass events [14]–[16]. These failures require fields that are not always retained by LLM-only tracing.

B. Boundary with prior AgentTelemetry work

This paper deliberately does not claim the novelty of the AgentTelemetry span taxonomy, fault classes, or SDK. Those belong to the AIware 2026 paper [7]. We cite that work as the source of the measurement artifacts and use its result files as input data. We also do not claim a new adapter architecture or runtime middleware layer. The contribution here is the data-quality analysis: what happens to structural diagnostic utility when trace schemas preserve or drop particular semantic fields.

This boundary matters for both scientific clarity and publication ethics. A benchmark/toolkit paper asks whether a set of faults and spans can be measured. A middleware paper asks how instrumentation can be attached to heterogeneous frameworks. This BigData paper asks a different question: once traces are produced, which schema properties are retained by the big-data pipeline for downstream analytics to remain valid?

III. Trace-Data Quality Model

We model an agent trace as a set of spans $T = \{s_1, \ldots, s_n\}$ with parent-child edges, timestamps, span kinds, and key-value attributes. A telemetry pipeline applies a schema mapping m that may retain spans, remove spans, collapse kinds, remove attributes, or preserve only structural fields. A data-quality evaluation must therefore distinguish *row retention* from *semantic preservation*: a trace may retain many rows while losing the columns needed for diagnosis.

A. Metrics

Schema coverage measures the fraction of observed semantic event kinds that remain visible after mapping:

$$C(m, T) = \frac{|K(m(T)) \cap K(T)|}{|K(T)|},$$

where $K(T)$ is the set of event kinds present in the original trace.

Span retention measures the fraction of spans that remain analytically typed after mapping:

$$R(m,T) = \frac{|\{s \in T : \mathrm{kind}(m(s)) \neq \bot\}|}{|T|}.$$

This differs from raw export count: a backend may store a span but strip the attribute that makes it usable for agent analytics.

Semantic preservation is a task-specific metric. For a diagnostic query q (for example, "find reasoning loops"), it asks whether the mapped trace contains the fields required to answer q. We use binary preservation for queries whose required fields are known: planning visibility, reasoning-loop visibility, guardrail event visibility, memory-operation visibility, and tool-error visibility.

Joinability measures whether events can be linked by trace identifier, parent-child relation, agent identifier, or task/session identifier. In this study we measure joinability qualitatively through diagnostic outcomes rather than a separate numeric score: if schema loss forces a diagnostic algorithm to inspect more candidate spans, joinability has degraded for that task.

Payload-redacted structural sufficiency asks whether structural fields are enough for a diagnostic task without prompt, completion, tool payload, reasoning-chain, or retrieved-query content. This metric does not claim that structural fields are risk-free; it asks whether a pipeline can remove high-sensitivity payload content while preserving planning, reasoning, memory, guardrail, and provenance structure.

Diagnostic sufficiency measures downstream utility: root-cause candidate set size, spans to diagnosis, signal-to-noise ratio, and fault-detection rate for fault families. These are not schema metrics by themselves; they are the effects that make schema quality operationally meaningful.

Table I makes the model operational. Each row is a downstream analytic query that a data platform might run over agent traces. The required fields are not exotic: event kind, parent relations, agent/task identifiers, and a few diagnostic attributes. The problem is that these fields cross schema layers. Some are produced by framework adapters, some are standardized by semantic conventions, some are preserved or dropped by collector processors, and some are redacted by privacy filters. A data-quality model must therefore follow the trace across the pipeline rather than treating export success as the end of the problem.

IV. Methodology

A. Data sources

We use existing open result artifacts from the AGENTTELEMETRY artifact package [8] and independent historical τ-bench trajectories [10]. No new LLM calls are required for the measurements in this paper; the new work is the schema-projection, backend-export, query-answerability, and field-ablation analysis over sanitized derived events.

SWE-bench trace corpus. The first corpus contains 112 SWE-bench agent traces with 3,060 spans. These traces include planning, memory, reasoning, LLM, retrieval, agent, tool, and guardrail events. We use this corpus to measure schema loss and diagnostic-proxy changes under degraded schema views.

E2E/topology trace suite. The second corpus contains 56 traces with 626 spans from end-to-end and multi-agent topology runs. Unlike the SWE-bench corpus, this suite includes explicit delegation events and broader topology structure. It gives the schema study an independent workload slice within the artifact package rather than only the controlled fault matrix.

τ-bench trajectory corpus. The third corpus contains 1,980 historical τ-bench trajectories with 42,581 derived events. Each trajectory has a task outcome and expected tool-action labels supplied by the benchmark. We convert each trajectory into a sanitized event trace containing an agent task event, assistant-call events, and tool-call events. The conversion records tool names, argument-shape counts, sequence order, and task outcome, but it does not write user messages, system prompts, tool outputs, or raw argument values into the paper artifacts. This workload is independent of the AGENTTELEMETRY artifact and supports an external tool-sequence visibility control.

Controlled fault matrix. The fourth result set is a 3,780-row fault-injection result matrix containing fourteen non-empty fault classes plus no-fault controls, six telemetry conditions, seven frameworks, and six LLMs. We use this matrix as validation for the fault-family interpretation, not as the center of the new contribution.

B. Schema views

We evaluate seven schema projections on the three trace corpora. The LLM-focused projections are mapped from public schema families: OTel GenAI semantic conventions (main branch f489c1832459), OpenInference semantic conventions (main branch 0ee46c6c0d8e), and OpenLLMetry v0.61.0 OTel application traces [2], [3], [6]. We map published span-kind and attribute families into the local normalized event vocabulary; a field is retained only when the cited convention exposes the corresponding span kind or attribute family, or when the projection is explicitly a custom-attribute or agent-structural view. Fields with no public anchor are counted absent. This makes the study a controlled field-retention analysis under the projection rules in Table III, not a benchmark of every deployment of those projects.

Vanilla OTel view. This conservative degradation removes the agent-specific analytical schema. It represents the case where spans may exist in a tracing backend but the downstream analyst cannot query agent span kinds or agent-specific fields.

OTel+custom-attributes view. This projection keeps the generic OTel span envelope and non-content diagnostic custom attributes while removing a dedicated event-kind column and payload content. It models a production team that keeps custom attributes inside a generic tracing backend.

GenAI-only view. This projection preserves LLM, tool, and retrieval fields corresponding to GenAI-oriented semantic conventions. It tests how far an LLM-centric schema can go without agent orchestration events.

TABLE I

DIAGNOSTIC QUERIES AND THE TRACE-DATA FIELDS THEY REQUIRE. THE TABLE ILLUSTRATES WHY ROW RETENTION ALONE IS NOT ENOUGH: A PIPELINE CAN EXPORT MANY SPANS WHILE DELETING THE FIELDS REQUIRED BY THE QUERY.

Diagnostic query	**Minimum trace-data requirements**	**Failure mode under schema loss**
Reasoning-loop detection	Reasoning event kind, agent/task identifier, iteration or parent-child order, token/cost fields when available	Repeated reasoning becomes a sequence of generic internal spans or LLM calls; loop detection falls back to brittle name matching or cannot run.
Guardrail audit	Guardrail event kind, policy/check identifier, decision outcome, relation to the triggering LLM/tool span	Blocked or bypassed safety events disappear from trace analytics; audit jobs cannot distinguish absent checks from passing checks.
Planning failure analysis	Planning event kind, plan representation or step count, parent agent, downstream tool/retrieval children	The analyst cannot connect an invalid action sequence to the plan that generated it; root-cause localization expands to many candidate spans.
Memory-corruption analysis	Memory event kind, operation, key or namespace, agent/session identity, relation to later reasoning/LLM spans	Stale or corrupted memory reads are indistinguishable from ordinary application lookups or are omitted entirely.
Tool-decision provenance	Tool-call event, parent LLM/reasoning relation, selected tool name, status/error, nearby retrieved context	Wrong-tool and tool-error analysis can see the tool call but not the reasoning path that selected it.

TABLE II

SANITIZED TRACE CORPORA USED BY THE BACKEND SCHEMA-QUALITY STUDY. THE τ-BENCH WORKLOAD IS INDEPENDENT OF THE AGENTTELEMETRY ARTIFACTS AND CONTRIBUTES TASK OUTCOMES PLUS EXPECTED TOOL-ACTION LABELS.

Corpus	Traces	Exported events	Event-kind distribution
SWE-bench traces	112	3060	AGENT:112, GUARD_RAIL:34, LLM_CALL:861, MEMORY:224, PLANNING:112, REASONING:914, RETRIEVAL:669, TOOL_CALL:134
E2E/topology traces	56	626	AGENT:176, DELEGATION:109, GUARD_RAIL:46, LLM_CALL:150, MEMORY:33, PLANNING:50, REASONING:38, RETRIEVAL:16, TOOL_CALL:8
τ-bench trajectories	1980	42581	AGENT:1980, LLM_CALL:26316, TOOL_CALL:14285

LLM+agent-run view. This projection preserves LLM/tool/retrieval, agent-run, and guardrail fields exposed by OpenInference-style span kinds. It does not preserve planning, memory, delegation, or separate reasoning-loop event structure.

LLM app view. This view retains LLM calls, tool calls, retrieval, and agent-level events while dropping planning, reasoning, guardrail, delegation, and memory semantics. We use it as an LLM-application trace schema-family mapping, not as a claim about every deployment of any specific system.

Agent structural view. This view preserves agent-specific event kinds and structural fields while redacting prompt, completion, tool payload, retrieved-query, reasoning-chain, and task content.

Agent reference view. This view retains all sanitized fields needed by the local artifacts and acts as the query-answerability reference. Raw payload content is not written to the paper directory.

C. Backend validation

Each projection is replayed through the OpenTelemetry SDK and exported to a SQLite trace-backend instance before metrics are computed. The backend has separate span and attribute tables and materializes 740,272 span rows plus 3,463,458 attribute rows across schema views and ablations. Representative aggregate queries are executed directly in SQL, and the query-answerability metrics are computed from backend rows rather than raw source records. This is not a benchmark of a production tracing backend. It validates the data-engineering path studied here: source events become spans, schema mappings determine visible fields, an exporter writes a backend representation, and downstream analytics run over that representation. The volume and velocity claims in this paper are therefore limited to controlled backend materialization and query execution, not production-scale ingestion.

As an additional backend-path check, an OTel Collector otelcol-contrib version 0.153.0 `otlpjsonfile→file` round trip over the OTel+custom-attributes projection preserved 46,267/46,267 spans, 258,617/258,617 diagnostic span attributes, and 44,119/44,119 parent links; the file exporter added 46,267 bookkeeping attributes that are excluded from the study queries.

For the controlled matrix, we use six telemetry conditions already present in the result file: no telemetry, vanilla OTel, OTel GenAI, OpenInference, structure-only agent traces, and full-capture agent traces.

D. Pipeline transformations

The analysis treats schema loss as a transformation problem. A trace pipeline can damage diagnostic quality in at least four ways.

Kind erasure maps semantically distinct events to a generic category. A reasoning step, planning step, and memory read may all survive as spans, but if they are indistinguishable `INTERNAL` events, downstream queries cannot use them as agent semantics.

Attribute stripping removes fields while preserving event kind. For example, a memory span without operation or key namespace may still count as a memory event but cannot support memory-corruption analysis. A guardrail span without outcome cannot support safety auditing.

Join-key loss breaks relations among spans. Agent analytics often need to connect a planning step to later tool calls, a

TABLE III

SCHEMA PROJECTIONS EVALUATED BY REPLAYING SANITIZED EVENTS THROUGH THE OPENTELEMETRY SDK AND A SQLITE TRACE BACKEND. PUBLIC ANCHORS ARE OTEL GENAI CONVENTIONS, OPENINFERENCE SPAN KINDS, AND OPENLLMETRY-STYLE OTEL APPLICATION TRACES; ROWS ARE FIELD-FAMILY MAPPINGS RATHER THAN PRODUCT-DEPLOYMENT MEASUREMENTS.

Projection	Public anchor	Retained field families	Absent field families
Vanilla OTel	OTel trace model	trace/span/parent IDs, timestamps, status	GenAI and agent diagnostic attributes
OTel+custom attrs	OTel custom span attributes	`agent.event_kind` and non-content diagnostic attributes	dedicated event-kind column and payload content
GenAI-only	OTel GenAI spans	`gen_ai.*` LLM/tool fields plus retrieval evidence	planning, reasoning, guardrail, delegation, memory, task labels
LLM+agent-run	OpenInference span kinds	`AGENT`, `LLM`, `TOOL`, `RETRIEVER`, `GUARDRAIL`	planning, reasoning loops, delegation, memory, tool-argument shape
LLM app	OpenLLMetry-style app traces	agent, LLM, tool, retrieval, token, and tool-shape fields	planning, reasoning, guardrail, delegation, memory
Agent structural	agent structural extension	all event kinds plus structural and non-content diagnostic fields	prompt/completion/reasoning/retrieval/tool payload content
Agent reference	local reference schema	all sanitized fields needed by the local artifacts	raw payload content

TABLE IV

SDK-TO-SQLITE BACKEND VALIDATION AND COST. THE FULL GENERATED BACKEND CONTAINS 740,272 SPAN ROWS AND 3,463,458 ATTRIBUTE ROWS ACROSS PROJECTIONS AND OCCUPIES 1178.9 MIB ON DISK. SQL MS IS THE COMBINED TIME FOR THREE BACKEND-NATIVE AGGREGATE QUERIES.

Projection	Spans	Attr rows	Export s	SQL ms	Eval s
Vanilla OTel	46267	0	1.15	0.5	0.84
OTel+custom attrs	46267	258617	1.95	21.7	1.25
GenAI-only	46267	169103	1.61	24.1	1.09
LLM+agent-run	46267	173799	1.53	23.9	1.06
LLM app	46267	187924	1.54	24.2	1.06
Agent structural	46267	258617	1.72	27.7	1.27
Agent reference	46267	259741	1.65	29.0	1.30

retrieval event to the LLM answer it grounded, or a memory read to a later reasoning span. Removing parent-child links, task identifiers, or agent identifiers turns a structured trace into an event bag.

Over-redaction treats content removal as schema deletion. Filtering should be able to remove prompt text, completion text, retrieved query content, tool payloads, reasoning-chain text, or memory values while retaining structural fields. If filtering deletes the span kind or join keys, it can destroy measured structural value unnecessarily.

E. Fault families

We separate fourteen fault classes into two families. *Operational* faults are detectable from generic execution, cost, timeout, or tool fields in this corpus: context overflow, cost explosion, infinite loop, timeout, tool failure, and wrong tool. *Semantic agent* faults require planning, reasoning, guardrail, delegation, retrieval freshness, or memory fields: agent misroute, circular delegation, guardrail bypass, hallucination, memory corruption, planning failure, reasoning loop, and stale retrieval. This split is intentionally diagnostic rather than taxonomic; it asks which fields are needed by the analysis pipeline.

F. Reproducibility

All tables in this paper are regenerated by scripts in the paper's `scripts/` directory. The default packaged path reads `generated/sanitized_events.jsonl`, `generated/tau_truth_labels.jsonl`, and `generated/trace_quality_metrics.json`; it then replays schema projections through the OpenTelemetry SDK, rebuilds the SQLite backend, runs structural visibility queries, applies field ablations, refreshes machine-readable metrics and LaTeX table snippets, and recompiles the PDF. An optional `--from-raw` mode rebuilds the sanitized events and summary metrics when the upstream result directories are available. The generated artifact manifest lists the generated table snippets and metric files, including the files used by the paper. Raw traces, conversation text, and tool payloads are not copied into the submission artifacts.

TABLE V

SCHEMA-INTEROPERABILITY METRICS OVER THE POOLED BACKEND. MACRO Q GIVES EACH POSITIVE STRUCTURAL QUERY FAMILY EQUAL WEIGHT; INST. Q POOLS QUERY INSTANCES AND IS DOMINATED BY HIGH-COUNT TOKEN/TOOL FAMILIES.

Projection	Kinds	Attrs	Macro Q	Inst. Q
Vanilla OTel	0.0%	0.0%	0.0%	0.00%
OTel+custom attrs	0.0%	100.0%	100.0%	100.00%
GenAI-only	33.3%	31.6%	37.5%	99.97%
LLM+agent-run	55.6%	47.4%	50.0%	99.97%
LLM app	44.4%	47.4%	37.5%	99.97%
Agent structural	100.0%	100.0%	100.0%	100.00%
Agent reference	100.0%	100.0%	100.0%	100.00%

V. RESULTS

*A. **RQ1**: LLM schemas preserve partial queryability*

Table V reports the schema-interoperability metrics over the pooled backend. The key result is not that generic schemas are useless; it is that they are selectively useful. The GenAI-only, LLM+agent-run, and LLM app projections preserve enough

TABLE VI
GROUND-TRUTH DENOMINATORS FOR STRUCTURAL VISIBILITY QUERIES IN THE REFERENCE BACKEND.

Query	Reference count
Loops	113
Plan	162
Guard	80
Memory	257
Deleg.	109
Tool prov.	14427
Tokens	2167286
Retr.	685

fields to support tool, token, retrieval, and τ-bench expected-action evidence. However, they retain only 33.3–55.6% of the semantic event kinds observed in the source corpora and answer only 37.5–50.0% of positive structural visibility query families overall. The OTel+custom-attributes projection is the counterexample: it has 0.0% event-kind column coverage, but it preserves 100.0% structural query recall because non-content diagnostic attributes remain queryable. The result is therefore not "OTel cannot work"; it is that the fields needed by these structural queries need to survive somewhere in the backend schema.

Metric choice is a visible sensitivity point rather than a hidden detail. Macro Q gives each diagnostic query family equal weight. If query instances are pooled, LLM-focused projections reach 99.97% recall because token-accounting contributes 2,167,286 of 2,183,119 positive instances. Over only the tool, token, and retrieval families, the LLM-focused projections score 100.0%; over the five orchestration families, GenAI-only and LLM app score 0.0%, while the OpenInference-style projection scores 20.0% by retaining guardrail spans. We therefore use Macro Q as the primary measure of diagnostic-family support and report Inst. Q as the throughput-weighted countermetric.

The distinction between raw trace availability and semantic availability is important. A tracing backend can report that an execution is fully traced while the exported schema prevents a downstream job from grouping spans by reasoning step, identifying the guardrail check that blocked execution, or joining memory reads to later model outputs. The quality defect is not missing storage; it is missing analytical structure.

Table VII makes the selectivity concrete. LLM-focused projections recover tool provenance, token-accounting, and retrieval-grounding queries; the OpenInference-style projection also recovers guardrail audit visibility. They do not recover planning, reasoning-loop, memory, or delegation queries. These failures are not row-loss artifacts. They are answerability failures: the projected trace table no longer contains the event kinds and attributes required by the query.

Table VIII separates this result by corpus. The τ-bench workload is easier for LLM-focused schemas because it is primarily a tool-calling workload. The smaller SWE-bench and E2E/topology corpora are more orchestration-heavy, and their recall drops to 37.5–42.9% for the LLM-focused projections.

The τ-bench outcome analysis is a negative control rather than a claim of strong outcome diagnosis. Missing expected actions are only a proxy for task failure: even with the reference schema, 5,880 of 8,880 expected actions are visible (66.2% action and argument-shape recall) and the proxy reaches 57.9% F1 against task pass/fail labels, barely above the always-fail F1 baseline. This deliberately weaker result shows that schema preservation makes tool-sequence and argument-shape evidence available, but task outcome diagnosis still needs domain semantics and argument-value validation.

B. *RQ2: Individual field classes are load-bearing*

The field-ablation study in Table XII separates schema design mistakes that a coarse projection table hides. Dropping both the event-kind column and event-kind custom attribute collapses query recall to 0.0% even though other attributes remain. Collapsing only orchestration event kinds drops query recall to 37.5%, matching the GenAI-only and LLM app projections in this study. Dropping only parent identifiers preserves event kinds and attributes but breaks tool provenance. Dropping only timestamps breaks reasoning-loop queries because repeated reasoning is an ordered phenomenon. Dropping only memory, guardrail, or token/cost attributes breaks the corresponding query family.

The positive result is narrower but still important: content redaction preserves 100.0% recall for the structural visibility queries in this workload. These queries require event kinds, ordering, join keys, and selected non-content attributes; they do not require prompt, completion, reasoning-chain text, tool payload, or retrieved query content. This does not mean content is unnecessary for every diagnostic task. Hallucination adjudication, stale-retrieval judgment, and argument-value debugging can require payload inspection. The result instead identifies the structural tasks that remain available under payload redaction, with ordinary access controls and retention policies still required for structural fields.

C. Validation against prior diagnostic proxies

The diagnostic-proxy effect is measurable. Table XIII compares agent-semantics traces with a degraded OTel view on failed SWE-bench traces. With agent semantics preserved, the mean root-cause candidate set is 3.0 spans; under degradation it is 9.5 spans. The mean number of spans inspected before the proxy diagnosis rises from 9.5 to 28.5. The trace still exists, but the missing schema forces the automated diagnostic procedure to search a broader, less structured execution space.

This is the practical meaning of trace-data quality for the measured LLM-agent diagnostics. Schema loss does not merely remove descriptive labels. It changes the cost of downstream structural analytics. A reasoning-loop detector cannot fire if reasoning spans are not represented. A guardrail audit cannot count blocked safety events if guardrail spans are flattened away. A memory-corruption analysis cannot connect a stale memory read to a later action if memory fields are absent.

The controlled fault matrix gives a second view of the same effect. Table XIV separates operational faults from semantic agent faults. Generic telemetry detects operational failures in

TABLE VII
STRUCTURAL VISIBILITY-QUERY RECALL BY SCHEMA PROJECTION. τ-BENCH ACTION AND OUTCOME EVIDENCE ARE REPORTED SEPARATELY IN TABLE IX.

Projection	Loops	Plan	Guard	Memory	Deleg.	Tool	Tokens	Retr.
Vanilla OTel	0.0%	0.0%	0.0%	0.0%	0.0%	0.0%	0.0%	0.0%
OTel+custom attrs	100.0%	100.0%	100.0%	100.0%	100.0%	100.0%	100.0%	100.0%
GenAI-only	0.0%	0.0%	0.0%	0.0%	0.0%	100.0%	100.0%	100.0%
LLM+agent-run	0.0%	0.0%	100.0%	0.0%	0.0%	100.0%	100.0%	100.0%
LLM app	0.0%	0.0%	0.0%	0.0%	0.0%	100.0%	100.0%	100.0%
Agent structural	100.0%	100.0%	100.0%	100.0%	100.0%	100.0%	100.0%	100.0%
Agent reference	100.0%	100.0%	100.0%	100.0%	100.0%	100.0%	100.0%	100.0%

TABLE VIII
PER-CORPUS STRUCTURAL QUERY RECALL. THIS SEPARATES THE INDEPENDENT τ-BENCH WORKLOAD FROM THE SMALLER ORCHESTRATION-HEAVY TRACE CORPORA.

Corpus	Pos. queries	GenAI-only	LLM app	Agent structural
SWE-bench	7	42.9%	42.9%	100.0%
E2E/topology	8	37.5%	37.5%	100.0%
τ-bench	2	100.0%	100.0%	100.0%

TABLE IX
τ-BENCH OUTCOME VALIDATION. MISSING EXPECTED ACTIONS ARE USED AS A SCHEMA-OBSERVABLE FAILURE PROXY AND EVALUATED AGAINST EXTERNAL TASK PASS/FAIL LABELS. THE FINAL COLUMN IS TP/FP/FN/TN FOR FAILURE PREDICTION.

Projection	Act.	Shape	Prec.	Recall	F1	Conf.
Vanilla OTel	0.0%	0.0%	41.3%	97.0%	57.9%	773/1099/24/84
OTel+custom attrs	66.2%	66.2%	54.9%	61.4%	57.9%	489/402/308/781
GenAI-only	66.2%	0.0%	54.9%	61.4%	57.9%	489/402/308/781
LLM+agent-run	66.2%	0.0%	54.9%	61.4%	57.9%	489/402/308/781
LLM app	66.2%	66.2%	54.9%	61.4%	57.9%	489/402/308/781
Agent structural	66.2%	66.2%	54.9%	61.4%	57.9%	489/402/308/781
Agent reference	66.2%	66.2%	54.9%	61.4%	57.9%	489/402/308/781

this corpus because those faults are exposed through generic execution signals such as timeout, cost, or tool error. It detects 0.0% of semantic agent faults because the required fields are absent. Structure-only and full-capture agent traces recover a subset of semantic-fault detections while retaining 100.0% operational-fault detection. The incomplete semantic detection rate is itself useful: schema preservation is necessary but not sufficient. Some agent faults also require framework support, expected-span specifications, or application-specific assertions. Table XV shows the class-wise pattern: circular delegation is almost entirely structural in this matrix, while most semantic classes need more than event-kind preservation.

This separation is useful for system design. If the only goal is to catch timeouts, tool exceptions, or cost spikes, generic LLM/tool telemetry may be enough. If the goal is to debug why an agent selected the wrong action, entered a reasoning loop, bypassed a guardrail, or used stale memory, generic telemetry has the wrong data model. A data platform should not advertise those diagnostic tasks unless it retains the corresponding schema fields.

TABLE X
τ-BENCH PASS/FAIL BASELINES FOR INTERPRETING THE MISSING-ACTION PROXY. THE FINAL COLUMN IS TP/FP/FN/TN.

Baseline	Acc.	Prec.	Recall	F1	Conf.
Always fail	40.3%	40.3%	100.0%	57.4%	797/1183/0/0
Always pass	59.7%	0.0%	0.0%	0.0%	0/0/797/1183
Tool-count mismatch	46.0%	42.0%	90.2%	57.3%	719/992/78/191

TABLE XI
PER-DOMAIN/MODEL τ-BENCH FAILURE-PROXY VALIDATION FOR THE REFERENCE SCHEMA. THE FINAL COLUMN IS TP/FP/FN/TN.

Group	Act.	Shape	Prec.	Recall	F1	Conf.
gpt-4o airline	67.1%	67.1%	75.9%	56.9%	65.0%	66/21/50/63
gpt-4o retail	62.8%	62.8%	54.7%	74.2%	62.9%	135/112/47/166
sonnet-35 airline	66.4%	66.4%	75.9%	46.8%	57.9%	101/32/115/152
sonnet-35 retail	67.7%	67.7%	44.1%	66.1%	52.9%	187/237/96/400

D. *RQ3: Calibration remains a separate risk*

However, schema quality does not remove calibration risk. Table XVI shows threshold sensitivity for selected detector parameters. Raising retry or token-growth thresholds reduces alert volume, but it can also erase a diagnostic class. For example, increasing the token-growth threshold from the default 1.3 to 1.45 reduces real-LLM alerts from 26 to 21 in the underlying result file. That reduction is not automatically a quality improvement; it may be a false-negative tradeoff. Trace-data quality therefore has two layers: the schema needs to preserve the fields needed by the target diagnostic query, and the analytics must calibrate thresholds without silently disabling the failure modes the schema made visible.

VI. DISCUSSION

A. Agent observability as data engineering

The results suggest that LLM-agent observability should be designed as a data pipeline, not just as an SDK feature. A production trace system needs a schema contract for agent events, collector transformations that do not erase orchestration semantics, storage schemas that retain join keys, and analytics that distinguish content capture from structural-field capture. Without those properties, a large trace lake can still be analytically low quality for agent diagnostics.

TABLE XII

FIELD-ABLATION STUDY ON THE REFERENCE SCHEMA. EACH ROW REMOVES ONE FIELD CLASS WHILE KEEPING THE REST OF THE BACKEND TABLE INTACT.

Ablation	Kinds	Attrs	Joins	Query recall	Failed positive queries
Drop semantic kind	0.0%	94.7%	100.0%	0.0%	reasoning_loops, planning_events, guardrail_audit, memory_ops, delegation_edges, tool_provenance, token_accounting, retrieval_grounding
Collapse orchestration	44.4%	94.7%	100.0%	37.5%	reasoning_loops, planning_events, guardrail_audit, memory_ops, delegation_edges
Drop parent IDs	100.0%	100.0%	0.0%	87.5%	tool_provenance
Drop timestamps	100.0%	100.0%	100.0%	87.5%	reasoning_loops
Drop agent IDs	100.0%	89.5%	100.0%	100.0%	none
Drop memory attrs	100.0%	89.5%	100.0%	87.5%	memory_ops
Drop guardrail attrs	100.0%	94.7%	100.0%	87.5%	guardrail_audit
Drop token/cost	100.0%	89.5%	100.0%	87.5%	token_accounting
Content redaction	100.0%	100.0%	100.0%	100.0%	none

TABLE XIII

DOWNSTREAM DIAGNOSTIC DEGRADATION WHEN AGENT-SPECIFIC TRACE DATA ARE REMOVED. LOWER IS BETTER FOR CANDIDATE SPANS AND SPANS-TO-DIAGNOSIS; HIGHER IS BETTER FOR SIGNAL-TO-NOISE.

Metric	AgentTelemetry	Degraded OTel	Effect
Candidate spans at root cause	3.0	9.5	68.4% fewer
Spans to diagnosis	9.5	28.5	66.8% fewer
Diagnostic signal-to-noise	0.54	0.00	semantic signal retained

TABLE XIV

CONTROLLED FAULT-DETECTION RESULTS SPLIT BY WHETHER THE FAULT REQUIRES AGENT-ORCHESTRATION SEMANTICS. OPERATIONAL FAULTS CAN OFTEN BE DETECTED FROM GENERIC LLM/TOOL/COST TRACES; SEMANTIC FAULTS DEGRADE SHARPLY WHEN PLANNING, REASONING, GUARDRAIL, DELEGATION, OR MEMORY FIELDS ARE ABSENT.

Telemetry condition	Semantic faults	Operational faults
No telemetry	0.0%	0.0%
Vanilla OTel	0.0%	100.0%
OTel GenAI	0.0%	100.0%
OpenInference	0.0%	100.0%
Structure-only	32.1%	100.0%
Full capture	32.1%	100.0%

This framing also explains why LLM-only observability is insufficient for agents. LLM calls are important rows in the trace table, but they are not the whole table. Agent diagnosis often depends on the rows around the LLM call: the plan that chose it, the reasoning step that interpreted it, the retrieval event that grounded it, the memory record that biased it, and the guardrail that approved or blocked it.

B. Payload-redacted structural visibility

The payload-redacted structural result is encouraging for a bounded class of diagnostics. Organizations do not always need to capture prompt, completion, retrieved-query, reasoning-chain, or tool-payload content to answer structural visibility questions about loops, guardrails, memory operations, delegation, token accounting, and tool provenance. Payload content remains necessary for other tasks, such as judging whether a retrieved document actually grounded an answer or whether a tool argument had the correct value. This does not eliminate sensitivity, because even structural fields can expose behavior and workflow shape, but it creates a better engineering tradeoff than all-or-nothing trace capture.

TABLE XV

CLASS-WISE CONTROLLED SEMANTIC-FAULT DETECTION. PRESERVING SCHEMA FIELDS IS NECESSARY BUT NOT SUFFICIENT: SEVERAL CLASSES ALSO REQUIRE APPLICATION ASSERTIONS, EXPECTED-SPAN SPECIFICATIONS, OR ARGUMENT/CONTENT CHECKS.

Fault class	OTel	OpenInf.	Struct.	Full
agent_misroute	0.0%	0.0%	14.3%	14.3%
circular_delegation	0.0%	0.0%	100.0%	100.0%
guardrail_bypass	0.0%	0.0%	14.3%	14.3%
hallucination	0.0%	0.0%	71.4%	71.4%
memory_corruption	0.0%	0.0%	14.3%	14.3%
planning_failure	0.0%	0.0%	14.3%	14.3%
reasoning_loop	0.0%	0.0%	14.3%	14.3%
stale_retrieval	0.0%	0.0%	14.3%	14.3%

TABLE XVI

CALIBRATION SENSITIVITY. TRACE-DATA SCHEMAS THAT PRESERVE AGENT SEMANTICS STILL REQUIRE THRESHOLD GOVERNANCE: RAISING RETRY OR TOKEN-GROWTH THRESHOLDS REDUCES ALERT VOLUME BUT CAN ERASE A DIAGNOSTIC CLASS. ASTERISKS MARK DEFAULTS.

Parameter	Value	SWE-bench alerts	Real-LLM alerts
max_retries	2	15	34
max_retries	3*	7	26
max_retries	5	3	6
token_growth_factor	1.15	25	59
token_growth_factor	1.3*	7	26
token_growth_factor	1.45	4	21
token_growth_factor	2.0	4	21

C. Why BigData is the right framing

The same empirical facts could be misframed as a tool comparison or an agent-debugging result. That would understate the broader systems lesson. The IEEE BigData framing is about the 5Vs of data-intensive systems. LLM-agent traces create volume in the form of many structured events, variety through heterogeneous producer schemas, velocity as traces move through collectors and backends, value through downstream diagnostic analytics, and veracity risks when schema transformations drop the fields that make trace rows meaningful. This paper demonstrates those risks on a controlled, backend-materialized corpus rather than a production ingestion benchmark. It therefore belongs with data and information quality, big-data infrastructure, and data-processing pipelines, not only with agent evaluation.

D. Implications for schema design

A useful agent-trace schema should satisfy four constraints. First, it should preserve agent-orchestration event kinds separately from generic internal spans. Second, it should make structural joins cheap by carrying trace, task, agent, parent, and session identifiers where appropriate. Third, it should allow privacy filters to redact payload content without deleting structural fields. Fourth, it should document the diagnostic tasks it supports. A schema that cannot answer whether a reasoning loop occurred or whether a guardrail fired should not be treated as complete agent observability.

E. Open implementation questions

Several engineering questions remain open. First, agent-trace schemas need versioning. A collector or warehouse should know whether a trace uses a generic, GenAI-only, LLM+agent-run, LLM app, or agent-specific extension, and should record which semantic fields were intentionally unavailable. Second, schema transformations should expose loss reports: if a processor drops reasoning or memory fields, that should be measurable provenance rather than an invisible side effect. Third, privacy filters should separate content sensitivity from structural sensitivity, allowing operators to keep diagnostic structure while redacting payloads. Finally, evaluations for observability systems should include downstream queries, not only ingestion throughput or export success.

VII. Threats to Validity

Schema degradation is an analytical model. The generic and LLM-focused views are controlled field-retention mappings from public schema families, not every possible deployment of a named system. A production team could add custom attributes to generic spans and recover some signal. That would support, rather than weaken, the paper's thesis: the data-quality outcome depends on which semantic fields are preserved.

The backend evidence is not a production benchmark. The OpenTelemetry SDK-to-SQLite path validates export, storage, and SQL querying over projected spans. The Collector round trip checks that one receiver/exporter path preserves spans, diagnostic attributes, and parent links for the custom-attribute projection, but it does not measure Jaeger, Tempo, ClickHouse, or search-backend behavior. The backend numbers should be read as trace-lake cost evidence for this study, not as production observability-system performance results.

The corpora are still limited. Two trace corpora and the controlled matrix come from the AGENTTELEMETRY artifact, while the τ-bench trajectory corpus is independent and uses external task labels. This improves independence but does not cover every agent framework, domain, or observability backend. Future work should repeat the same schema-loss analysis on additional production trace corpora and backend deployments.

Semantic faults are harder than operational faults. The controlled matrix shows that even full-capture and structure-only agent traces do not detect all semantic faults in all frameworks. This is not hidden in the analysis. It means schema preservation is a necessary condition for semantic fault diagnosis, not a complete solution.

Diagnostic metrics are proxies. Candidate spans, spans to diagnosis, and signal-to-noise are automated proxies for diagnostic effort. They do not replace a human operator study. They are appropriate for this paper because the question is data-quality degradation in the trace pipeline, not human incident response performance.

The paper uses prior artifacts. The paper intentionally builds on an accepted prior artifact. The risk is perceived overlap. We mitigate that by citing the prior work as the source of the span/fault definitions, avoiding benchmark or toolkit novelty claims, and limiting the new claims to schema-loss, data-quality, and diagnostic-degradation analysis.

VIII. Related Work

Distributed tracing and observability. Classical tracing systems such as Dapper, X-Trace, and Magpie established the value of structured request traces for large-scale systems [11]–[13]. OTel generalized these ideas into vendor-neutral APIs, SDKs, collectors, and semantic conventions for modern production systems [1]. This paper studies a new schema-quality problem that arises when the traced computation is an LLM-agent workflow rather than a service call graph.

Data quality, schema interoperability, and provenance. Data-quality research distinguishes syntactic correctness from fitness for consumer tasks [17], [18]. Schema-matching and data-integration work studies how heterogeneous schemas can be aligned and where mappings fail [19]. Database provenance work shows why downstream query answers depend on which source records and transformations survive the pipeline [20]. This paper applies those ideas to observability traces: a span table is not high quality merely because rows are present; it is high quality only if it preserves the fields needed by diagnostic consumers.

LLM and agent observability. OTel GenAI, OpenInference, OpenLLMetry, LangSmith, and Langfuse all address parts of LLM observability [2]–[6]. Their schemas make LLM calls, tool calls, retrieval, and application traces easier to analyze. Our analysis shows why agent-level orchestration semantics are needed by the measured diagnostic workloads.

Agent failure analysis. Recent work documents multi-agent and LLM-agent failure modes, debugging workflows, and observability needs [14]–[16]. The prior AGENTTELEMETRY paper introduced a benchmark and toolkit for agent fault detection [7]. τ-bench provides a complementary benchmark for tool-agent-user interactions with task outcomes and expected actions [10]. This paper studies the data-quality and schema-loss properties of telemetry pipelines that carry such traces.

IX. Conclusion

LLM-agent observability is a big-data quality problem. A pipeline can store thousands of spans while losing the semantic structure needed for diagnosis. Across 2,148 traces and 46,267 backend-exported events, LLM-focused schema projections preserve tool, token, and retrieval query families but answer

only 37.5–50.0% of positive structural visibility query families overall and 0.0% of the measured planning, reasoning-loop, memory, and delegation families. Field ablations show that event kind, parent identifiers, timestamps, memory fields, guardrail outcomes, and token/cost fields each carry distinct diagnostic load. A payload-redacted structural projection preserves all measured structural visibility answers without retaining prompt, completion, reasoning-chain, retrieved-query, or tool payload content. A generic OTel span store can also preserve these structural queries when non-content custom diagnostic attributes survive export. Agent traces therefore need explicit schema-quality contracts: if planning, reasoning, guardrail, delegation, and memory fields do not survive ingestion and transformation, analytics can run on complete-looking traces while answering incomplete questions.

REFERENCES

[1] OpenTelemetry Authors, "OpenTelemetry Specification," 2024. [Online]. Available: https://opentelemetry.io/docs/specs/otel/

[2] ——, "OpenTelemetry semantic conventions for GenAI operations," https://opentelemetry.io/docs/specs/semconv/gen-ai/, 2025, f489c1832459, Jun. 2026.

[3] Traceloop, "OpenLLMetry: Open-source observability for LLM applications," https://github.com/traceloop/openllmetry, 2025, v0.61.0, Jun. 2026.

[4] LangChain, Inc., "LangSmith: Platform for LLM application development, monitoring, and testing," https://smith.langchain.com, 2025.

[5] Langfuse, "Langfuse: Open-source LLM engineering platform," https://github.com/langfuse/langfuse, 2025.

[6] Arize AI, "OpenInference: Semantic conventions for LLM, ml, and agent observability," 2024, 0ee46c6c0d8e, Jun. 2026. [Online]. Available: https://github.com/Arize-ai/openinference

[7] K. C. Balusu, "AgentTelemetry: A fault detection benchmark and toolkit for LLM agent observability," in *Proceedings of the 3rd ACM International Conference on AI-Powered Software (AIware '26)*. ACM, 2026, montreal, QC, Canada, July 6–7, 2026.

[8] ——, "AgentTelemetry: Reproduction artifact," 2026.

[9] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. Narasimhan, "SWE-bench: Can language models resolve real-world GitHub issues?" in *Proceedings of the 12th International Conference on Learning Representations*, 2024. [Online]. Available: https://openreview.net/forum?id=VTF8yNQM66

[10] S. Yao, N. Shinn, P. Razavi, and K. Narasimhan, "τ-bench: A benchmark for tool-agent-user interaction in real-world domains," 2024. [Online]. Available: https://arxiv.org/abs/2406.12045

[11] B. H. Sigelman, L. A. Barroso, M. Burrows, P. Stephenson, M. Plakal, D. Beaver, S. Jaspan, and C. Shanbhag, "Dapper, a Large-Scale Distributed Systems Tracing Infrastructure," Google, Tech. Rep. dapper-2010-1, 2010. [Online]. Available: https://research.google/pubs/dapper-a-large-scale-distributed-systems-tracing-infrastructure/

[12] R. Fonseca, G. Porter, R. H. Katz, S. Shenker, and I. Stoica, "X-Trace: A pervasive network tracing framework," in *Proceedings of the 4th USENIX Symposium on Networked Systems Design and Implementation*, 2007. [Online]. Available: https://www.usenix.org/legacy/event/nsdi07/tech/fonseca.html

[13] P. Barham, A. Donnelly, R. Isaacs, and R. Mortier, "Using magpie for request extraction and workload modelling," in *Proceedings of the 6th Symposium on Operating Systems Design and Implementation*, 2004. [Online]. Available: https://www.usenix.org/legacy/event/osdi04/tech/barham.html

[14] M. Cemri, M. Z. Pan, S. Yang, L. A. Agrawal, B. Chopra, R. Tiwari, K. Keutzer, A. Parameswaran, D. Klein, K. Ramchandran, M. Zaharia, J. E. Gonzalez, and I. Stoica, "Why do multi-agent LLM systems fail?" *arXiv preprint arXiv:2503.13657*, 2025.

[15] L. Dong, Q. Lu, and L. Zhu, "AgentOps: Enabling observability of LLM agents," 2024.

[16] W. Epperson, G. Bansal, V. Dibia, A. Fourney, J. Gerrits, E. Zhu, and S. Amershi, "Interactive debugging and steering of multi-agent AI systems," in *Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems*, 2025.

[17] R. Y. Wang and D. M. Strong, "Beyond accuracy: What data quality means to data consumers," *Journal of Management Information Systems*, vol. 12, no. 4, pp. 5–33, 1996.

[18] C. Batini, C. Cappiello, C. Francalanci, and A. Maurino, "Methodologies for data quality assessment and improvement," *ACM Computing Surveys*, vol. 41, no. 3, pp. 1–52, 2009.

[19] E. Rahm and P. A. Bernstein, "A survey of approaches to automatic schema matching," *The VLDB Journal*, vol. 10, no. 4, pp. 334–350, 2001.

[20] J. Cheney, L. Chiticariu, and W.-C. Tan, "Provenance in databases: Why, how, and where," *Foundations and Trends in Databases*, vol. 1, no. 4, pp. 379–474, 2009.